

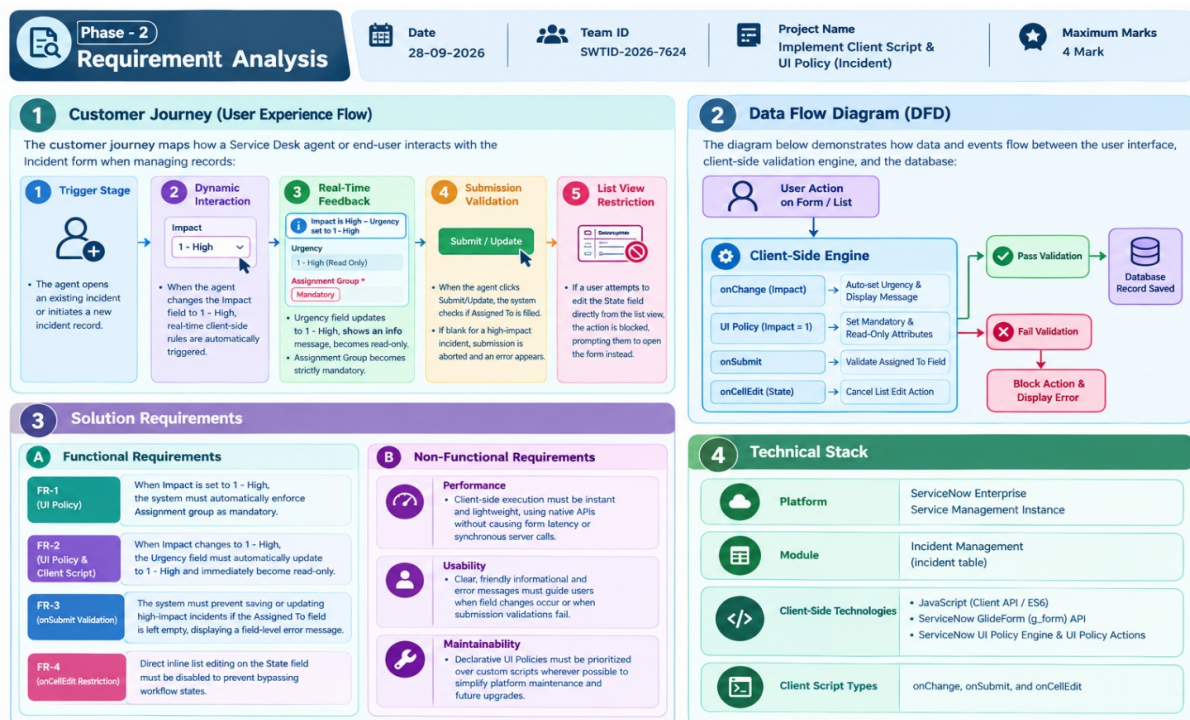
Phase-2 Requirement Analysis

Field	Details
Date	28-09-2026
Team ID	SWTID-2026-8733
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Mark

Requirement Analysis

Requirement Analysis is the second critical phase of the project development lifecycle. Building upon the concepts identified during brainstorming, this phase clearly specifies the functional and technical requirements needed to implement dynamic client-side controls on the Incident form.

To ensure data integrity, streamline user experience, and prevent human errors during incident creation and updates, the following core requirements were analyzed:



1. Executive Summary & Overview

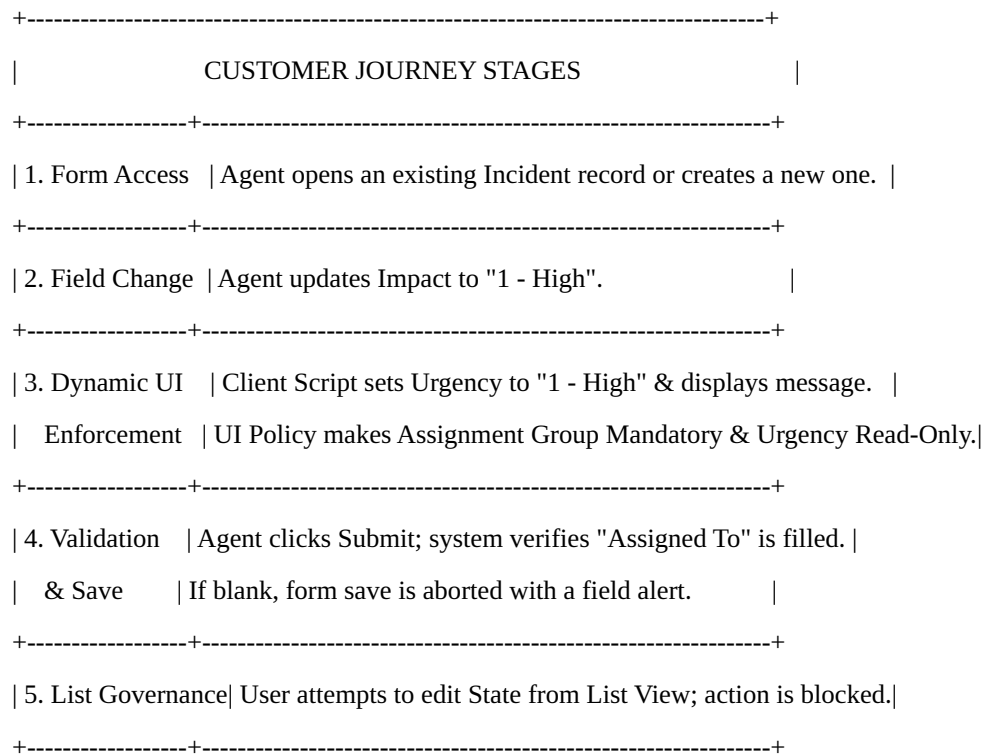
The **Requirement Analysis Phase** translates the high-level concepts established during brainstorming into concrete, structured functional and technical specifications. In ServiceNow Incident Management, enforcing

proper form logic and data validation on the client side reduces invalid submissions and improves Service Level Agreement (SLA) compliance.

This phase defines the end-to-end user journey, maps system data flows, specifies functional and non-functional requirements, and establishes the exact technical stack required to deploy Client Scripts and UI Policies on the incident table.

2. Customer Journey (User Experience Flow)

The customer journey illustrates how a Service Desk Agent interacts with the Incident form when logging or modifying critical incidents:



3. Data Flow Diagram (DFD)

The Data Flow Diagram models how data triggers move through client-side engines before committing records to the ServiceNow database:

[Service Desk Agent Input]

|



[Client-Side Engine]

|——> Trigger: onChange (Impact) ——> Auto-set Urgency & Display Message

|——> Trigger: UI Policy (Impact=1) —> Set Mandatory & Read-Only Attributes

|——> Trigger: onSubmit ——> Validate Assigned To Field

|——> Trigger: onCellEdit (State) ——> Block Direct List Edit

|

|—— Pass Validation ——> [ServiceNow Database (incident)]

|

|—— Fail Validation ——> [Display Error & Abort Operation]

4. Solution Requirements Specification

A. Functional Requirements

- **FR-01: Dynamic Mandatory Assignment Group (UI Policy)** When Impact is set to 1 - High, the Assignment group field must dynamically become mandatory. When the condition is no longer met, the rule must automatically revert (Reverse if false = true).
- **FR-02: Urgency Field Automation & Protection (UI Policy & Client Script)**
 - When Impact changes to 1 - High, an onChange Client Script must automatically update Urgency to 1 - High and display an informational banner (g_form.addInfoMessage).
 - A UI Policy must concurrently set the Urgency field to Read-Only to prevent manual override.
- **FR-03: Mandatory Assignment Validation on Submit (Client Script)** An onSubmit Client Script must check if Assigned To is empty when Impact is 1 - High. If empty, it must stop form submission (return false) and highlight the missing field (g_form.showErrorMessage).
- **FR-04: List Editing Restriction on State Field (Client Script)** An onCellEdit Client Script must intercept any direct inline list editing on the State field, alert the user to use the form view, and cancel the edit (callback(false)).

B. Non-Functional Requirements

- **Performance:** All client-side scripts must execute instantly without relying on synchronous server queries (GlideRecord) to preserve form rendering speed.
- **Usability:** Feedback messages (error boxes, info banners) must be clear, actionable, and non-intrusive.
- **Maintainability & Security:** UI Policies must be used for declarative UI changes to lower technical debt, while client scripts are protected against direct list bypasses.

5. Technical Stack

Category	Declarative UI Engine	Purpose	
Platform	Client Script Engine	Core Cloud ITSM Platform	
Target Table	Client API	Incident Management Data Model	
Declarative UI Engine	UI Engine	Dynamic field state changes (Mandatory / Read-Only)	
Client Script Engine	Client Script Engine Declarative	Client-side logical validation, auto-population, and list control[cite: 2]	
Client API	Client API	Field manipulation, error messaging, and submission abortion[cite: 2]	